

**MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Temciuc Adelina FAF-242

Report

on laboratory work no. 1
Cryptography and Security
Topic: The Caesar Cipher

Checked by:
Zaica Maia, university assistant
DISA, FCIM, UTM

Chişinău – 2026

AIM OF THE WORK

Purpose of the laboratory work

The purpose of this laboratory work is to understand how the Caesar cipher works, to see why its small key space makes it insecure, to strengthen it with a keyword-based permutation of the alphabet, and to implement both versions for the Romanian alphabet of 31 letters.

The laboratory work aims to:

- implement the classic Caesar cipher (Task 1.1) for the Romanian alphabet, using only the letter codes $A = 0, \text{Ă} = 1, \dots, Z = 30$;
- implement the Caesar cipher with two keys (Task 1.2): a numeric shift and a keyword that permutes the alphabet;
- validate every input (the key, the keyword and the text) and ask again with a clear message when it is wrong;
- compare the key space and the security of the two versions.

THEORETICAL BACKGROUND

The Caesar cipher

In the Caesar cipher every letter of the plaintext is replaced by the letter found a fixed number of positions further along the alphabet. The secret key k is this shift, and it is the same for encryption and decryption. If x and y are the numeric codes of a plaintext and a ciphertext letter and n is the length of the alphabet, then:

$$\text{encryption: } c = (x + k) \bmod n, \quad \text{decryption: } m = (y - k) \bmod n.$$

When $y - k$ is negative, n is added to bring the result back into the range $0 \dots n - 1$. The key 0 is excluded because it leaves the text unchanged, so $k \in \{1, 2, \dots, n - 1\}$. For the Romanian alphabet $n = 31$, so there are only 30 possible keys. A cryptanalyst can simply try all of them (a brute-force attack): exactly one key produces readable Romanian text.

The Caesar cipher with a permutation

To make the key space larger, a second key is used: a keyword k_2 . The alphabet is rewritten by placing the letters of the keyword first, each only once, followed by the remaining letters in their natural order. The shift k_1 is then applied with the formulas above, but the code of a letter is now its position in the permuted alphabet.

The number of possible orderings of the Romanian alphabet is $31! \approx 8.22 \times 10^{33}$, so the number of key pairs (k_1, k_2) is

$$31! \times 30 = 246\,685\,159\,625\,337\,684\,531\,766\,886\,400\,000\,000 \approx 2.47 \times 10^{35}.$$

This makes exhaustive search impossible in practice, but the cipher is still a monoalphabetic substitution: each plaintext letter is always replaced by the same ciphertext letter, so the letter frequencies of the language remain visible and the cipher can be broken by frequency analysis.

PART 1. IMPLEMENTATION

Both tasks were implemented in Python 3 without external libraries. The program is a console application: the user chooses the task and the operation, enters the keys and the text, and the result is printed in the terminal. To keep each file small and focused on one job, the code is split into four modules:

- `alphabet.py` – the Romanian alphabet, the letter codes and the permuted alphabet;
- `validation.py` – checking the key, the keyword and the text;
- `cipher.py` – encryption and decryption;
- `main.py` – the menu and the dialogue with the user; the program is started with `python3 main.py`.

`main.py` imports the other three modules, `validation.py` and `cipher.py` use only `alphabet.py`, and the cipher functions know nothing about the console, so they can be reused later, for example in a graphical interface.

1.1 The letter encoding

In `alphabet.py` the alphabet is stored as a list in the order of Table 1.1, so the code of a letter is simply its index in the list. A second list holds the lowercase letters, and a dictionary maps every accepted character to its uppercase letter. The cedilla forms Ș and Ț, which some keyboards produce, are mapped to the correct letters Ș and Ț.

Table 1.1 – Letter encoding of the Romanian alphabet (n = 31)

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M
x	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	
m	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z	

```
# Table 2: the Romanian alphabet (n = 31), letter codes A = 0 ... Z = 30.
# The code of a letter is its index in this list.
ALPHABET = list("ĂĂABCDEFHGHIÎJKLMNOPSȘTȚUVWXYZ")
LOWER = list("ăăâbcdefghiîjklmnopqrsștțuvwxyz")
N = len(ALPHABET)
```

```
# Every accepted character mapped to its uppercase letter. The cedilla forms
# Ș/ș and Ț/ț are treated as the comma-below letters Ș and Ț.
TO_UPPER = {}
for upper, lower in zip(ALPHABET, LOWER):
    TO_UPPER[upper] = upper
    TO_UPPER[lower] = upper
TO_UPPER.update({"Ș": "Ș", "ș": "Ș", "Ț": "Ț", "ț": "Ț"})
```

No ASCII or Unicode code point is used to compute the shift: the codes come only from the position of the letter in the list. The uppercase conversion also uses the dictionary, so a character that is not in the Romanian alphabet can never slip through.

1.2 Input validation

The module `validation.py` has a separate function for each input, which either returns a clean value or raises `InputError` with a message stating the allowed values. The key must be an integer from 1 to 30:

```
def parse_key(raw):
    """Key 1 must be an integer from 1 to N - 1 (1..30)."""
    raw = raw.strip()
    if not raw.lstrip("-").isdigit():
        raise InputError(f"'{raw}' is not an integer. {KEY_RULE}")
    key = int(raw)
    if not 1 <= key <= N - 1:
        raise InputError(f"{key} is out of range. {KEY_RULE}")
    return key
```

The text may contain only Romanian letters (in any case) and spaces. The spaces are removed, the letters are converted to uppercase, and the first rejected character is reported together with its position:

```
def prepare_text(raw):
    """Check the characters, convert to uppercase and remove the spaces."""
    raw = unicodedata.normalize("NFC", raw)
    result = []
    for position, ch in enumerate(raw, start=1):
        if ch == " ":
            continue
        if ch not in TO_UPPER:
            raise InputError(
                f"Invalid character '{ch}' at position {position}.\n"
                f"Only Romanian letters ({ALLOWED_LETTERS}) and spaces "
                f"are allowed.")
        result.append(TO_UPPER[ch])
    if not result:
        raise InputError("The text is empty. Enter at least one letter.")
    return "".join(result)
```

The keyword follows the same rules, but spaces are not allowed and it must have at least 7 letters:

```
def prepare_keyword(raw):
    """Key 2: only Romanian letters, at least 7, converted to uppercase."""
    raw = unicodedata.normalize("NFC", raw.strip())
    result = []
    for position, ch in enumerate(raw, start=1):
        if ch not in TO_UPPER:
            shown = "space" if ch == " " else f"'{ch}'"
            raise InputError(
                f"Invalid character {shown} at position {position}.\n"
                f"The keyword may contain only Romanian letters "
                f"({ALLOWED_LETTERS}).")
        result.append(TO_UPPER[ch])
    if len(result) < 7:
        raise InputError(f"The keyword has {len(result)} letters. "
                        f"It must have at least 7 letters.")
    return "".join(result)
```

The function `ask()` in `main.py` repeats the question until the input is accepted, printing the error each time, so the user never has to restart the program:

```
def ask(prompt, parser):
    """Ask again and again until the parser accepts the value."""
    while True:
        try:
            return parser(input(prompt))
        except InputError as error:
            print("  Error: " + str(error).replace("\n", "\n          "))
```

1.3 Encryption and decryption

Both operations are in `cipher.py` and use one function, `shift()`. It builds a dictionary from letter to code for the alphabet it receives, adds the key modulo n and takes the letter at the new position. Decryption is the same shift with the opposite key, which gives exactly $m = (y - k) \bmod n$. In Python the operator `%` never returns a negative value, but the check for a negative result is kept to match the formula from the theory:

```
def shift(text, key, alphabet=ALPHABET):
    """Replace every letter x by the letter with code (x + key) mod n.

    The codes come from the position of the letter in `alphabet`, never from
    ASCII/Unicode values. A negative key gives decryption.
    """
    code = {letter: index for index, letter in enumerate(alphabet)}
    n = len(alphabet)
    out = []
    for letter in text:
        y = (code[letter] + key) % n
        if y < 0:          # Python's % is never negative, kept for clarity
            y += n
        out.append(alphabet[y])
    return "".join(out)

def encrypt(text, key, alphabet=ALPHABET):
    return shift(text, key, alphabet)          # c = (x + k) mod n

def decrypt(text, key, alphabet=ALPHABET):
    return shift(text, -key, alphabet)         # m = (y - k) mod n
```

Because the alphabet is a parameter, the same functions serve both tasks: Task 1.1 passes the normal alphabet, and Task 1.2 passes the permuted one.

1.4 Building the permuted alphabet

The function `permuted_alphabet()` in `alphabet.py` adds the whole alphabet after the keyword, and the letters are added one by one, skipping those already present. This keeps only the first occurrence of each letter and always produces all 31 letters:

```
def permuted_alphabet(keyword):
    """Keyword letters first (each once), then the rest in natural order."""
    order = []
    for letter in keyword + "".join(ALPHABET):
        if letter not in order:
            order.append(letter)
    return order
```

For the keyword `criptografie` (uppercase `CRIPTOGRAFIE`, 10 distinct letters C, R, I, P, T, O, G, A, F, E – the second R and the second I are dropped) the permuted alphabet is `CRIPTOGA FEA ĀĀBDHÎJKLMNQSSŢUVWXYZ`. The program prints it so that the result can be checked by hand, as shown in Table 1.2 for $k_1 = 3$.

Table 1.2 – Caesar cipher with $k_1 = 3$ and $k_2 = \text{CRIPTOGRAFIE}$

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	C	R	I	P	T	O	G	A	F	E	Ă	Â	B	D	H	Î
x+k	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
c	P	T	O	G	A	F	E	Ă	Â	B	D	H	Î	J	K	L
x	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	
m	J	K	L	M	N	Q	S	Ş	Ţ	U	V	W	X	Y	Z	
x+k	19	20	21	22	23	24	25	26	27	28	29	30	0	1	2	
c	M	N	Q	S	Ş	Ţ	U	V	W	X	Y	Z	C	R	I	

1.5 The command-line interface

The module `main.py` holds the dialogue with the user. The main menu offers the two tasks and an exit option. `run_task()` asks for the operation, the key (and for Task 1.2 the keyword, after which it prints both alphabets), then the message or the ciphertext, and prints the prepared text and the result:

```
def run_task(two_keys):
    operation = ask_choice("Operation - (e)ncrypt or (d)ecrypt: ", ["e", "d"])
    label = "Key 1" if two_keys else "Key"
    key = ask(f"{label} (integer 1-{N - 1}): ", parse_key)

    alphabet = ALPHABET
    if two_keys:
        keyword = ask("Key 2 (keyword, at least 7 Romanian letters): ",
                      prepare_keyword)
        alphabet = permuted_alphabet(keyword)
        print(f"Keyword: {keyword}")
        print(f"Original alphabet: {''.join(ALPHABET)}")
        print(f"Permuted alphabet: {''.join(alphabet)}")

    if operation == "e":
        text = ask("Message: ", prepare_text)
        print(f"Prepared message: {text}")
        print(f"Ciphertext: {encrypt(text, key, alphabet)}")
    else:
        text = ask("Ciphertext: ", prepare_text)
        print(f"Prepared text: {text}")
        print(f"Decrypted message: {decrypt(text, key, alphabet)}")
```

PART 2. RESULTS

2.1 Task 1.1 – the Caesar cipher

The message „Învăţăm cifrul Cezar” was encrypted with the key $k = 3$. The program converted it to uppercase and removed the spaces, giving ÎNVĂȚĂMCIFRULCEZAR, and printed the ciphertext LQYCWCPFKITXOFHÂBT. For example, Î has the code 11 and becomes the letter with code 14, which is L; Z (code 30) becomes the letter with code $(30 + 3) \bmod 31 = 2$, which is Â.

```
student@ubuntu:~/caesar-lab1$ python3 main.py
=== Caesar cipher - Romanian alphabet (n = 31) ===

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 1
Operation - (e)ncrypt or (d)ecrypt: e
Key (integer 1-30): 3
Message: Învăţăm cifrul Cezar
Prepared message: ÎNVĂȚĂMCIFRULCEZAR
Ciphertext:      LQYCWCPFKITXOFHÂBT

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 0
Bye!
student@ubuntu:~/caesar-lab1$ █
```

Figure 2.1 – Encryption with the Caesar cipher, $k = 3$

The ciphertext was then decrypted with the same key, and the original message was obtained:

```

student@ubuntu:~/caesar-lab1$ python3 main.py
=== Caesar cipher - Romanian alphabet (n = 31) ===

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 1
Operation - (e)ncrypt or (d)ecrypt: d
Key (integer 1-30): 3
Ciphertext: LQYCWCPFKITXOFHÂBT
Prepared text: LQYCWCPFKITXOFHÂBT
Decrypted message: ÎNVĂȚĂMCIFRULCEZAR

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 0
Bye!
student@ubuntu:~/caesar-lab1$ █

```

Figure 2.2 – Decryption with the Caesar cipher, $k = 3$

Figure 2.3 shows the messages for invalid input: a wrong operation, the keys 0 and 31, which are out of range, the word „trei”, which is not an integer, and a message with digits and „!”. After each error the program asked again. The last message was typed with the cedilla letters Ș and Ț, which were accepted as Ș and Ț.

```

student@ubuntu:~/caesar-lab1$ python3 main.py
=== Caesar cipher - Romanian alphabet (n = 31) ===

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 1
Operation - (e)ncrypt or (d)ecrypt: x
Error: choose one of: e, d.
Operation - (e)ncrypt or (d)ecrypt: e
Key (integer 1-30): 0
Error: 0 is out of range. The key must be an integer from 1 to 30.
Key (integer 1-30): 31
Error: 31 is out of range. The key must be an integer from 1 to 30.
Key (integer 1-30): trei
Error: 'trei' is not an integer. The key must be an integer from 1 to 30.
Key (integer 1-30): 7
Message: Salut 2026!
Error: Invalid character '2' at position 7.
Only Romanian letters (A-Z, a-z, Ă ă, Â â, Î î, Ș ș, Ț ț) and spaces are allowed.
Message: Știință și artă
Prepared message: ȘȚIINȚĂȘIARTĂ
Ciphertext: YZOOTAGYOFWZG

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 0
Bye!
student@ubuntu:~/caesar-lab1$ █

```

Figure 2.3 – Invalid input in Task 1.1

2.2 Task 1.2 – the Caesar cipher with a permutation

The same message was encrypted with $k_1 = 3$ and $k_2 = \text{criptografie}$. The program printed the permuted alphabet and the ciphertext LŞYDWDSPOÂTXQPBIĂT, which is different from the one of Task 1.1 (LQYCWCPFKITXOFHÂBT) although the shift is the same.

```
student@ubuntu:~/caesar-lab1$ python3 main.py
=== Caesar cipher - Romanian alphabet (n = 31) ===

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 2
Operation - (e)ncrypt or (d)ecrypt: e
Key 1 (integer 1-30): 3
Key 2 (keyword, at least 7 Romanian letters): criptografie
Keyword:          CRIPTOGRAFIE
Original alphabet: AĂÂBCDEFGHIÎJKLMNOPSŞŢTUVWXYZ
Permuted alphabet: CRIPTOGAFEAĂBDHIÎJKLMNQSSŢTUVWXYZ
Message: Învăţăm cifrul Cezar
Prepared message: ÎNVĂŢĂMCIFRULCEZAR
Ciphertext:      LŞYDWDSPOÂTXQPBIĂT

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 0
Bye!
student@ubuntu:~/caesar-lab1$
```

Figure 2.4 – Encryption with two keys, $k_1 = 3$, $k_2 = \text{criptografie}$

Decryption with the same two keys (the keyword typed this time as „Criptografie”) gave back the original message:

```

student@ubuntu:~/caesar-lab1$ python3 main.py
=== Caesar cipher - Romanian alphabet (n = 31) ===

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 2
Operation - (e)ncrypt or (d)ecrypt: d
Key 1 (integer 1-30): 3
Key 2 (keyword, at least 7 Romanian letters): Criptografie
Keyword:          CRIPTOGRAFIE
Original alphabet: AĂÂBCDEFGHIÎJKLMNOPSŞŢTUVWXYZ
Permuted alphabet: CRIPTOGAFEAĂBDHIÎJKLMNQSSŢTUVWXYZ
Ciphertext: LŞYDWDSPOĂTXQPBIĂT
Prepared text:    LŞYDWDSPOĂTXQPBIĂT
Decrypted message: ÎNVĂŢĂMCIFRULCEZAR

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 0
Bye!
student@ubuntu:~/caesar-lab1$ █

```

Figure 2.5 – Decryption with two keys

Figure 2.6 shows the validation of both keys: the key 45 is out of range, „cheie” has only 5 letters, „parola123” contains digits and „cheie secretă” contains a space. After a correct keyword, a message with a digit was rejected, and the corrected message was encrypted.

```

student@ubuntu:~/caesar-lab1$ python3 main.py
=== Caesar cipher - Romanian alphabet (n = 31) ===

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 2
Operation - (e)ncrypt or (d)ecrypt: e
Key 1 (integer 1-30): 45
    Error: 45 is out of range. The key must be an integer from 1 to 30.
Key 1 (integer 1-30): 5
Key 2 (keyword, at least 7 Romanian letters): cheie
    Error: The keyword has 5 letters. It must have at least 7 letters.
Key 2 (keyword, at least 7 Romanian letters): parola123
    Error: Invalid character '1' at position 7.
        The keyword may contain only Romanian letters (A-Z, a-z, Ă ă, Â â, Î î, Ș ș, Ț ț).
Key 2 (keyword, at least 7 Romanian letters): cheie secretă
    Error: Invalid character space at position 6.
        The keyword may contain only Romanian letters (A-Z, a-z, Ă ă, Â â, Î î, Ș ș, Ț ț).
Key 2 (keyword, at least 7 Romanian letters): securitate
Keyword:          SECURITATE
Original alphabet: AĂÂBCDEFGHIÎJKLMNOPSȘȚTUVWXYZ
Permuted alphabet: SECURITAĂÂBDFGHÎJKLMNOPSȚTUVWXYZ
Message: atac la ora 5
    Error: Invalid character '5' at position 13.
        Only Romanian letters (A-Z, a-z, Ă ă, Â â, Î î, Ș ș, Ț ț) and spaces are allowed.
Message: atac la ora cinci
Prepared message: ATACLAORACINCI
Ciphertext:      FDFAQFVÂFABȚAB

1. Caesar cipher (Task 1.1)
2. Caesar cipher with a keyword permutation (Task 1.2)
0. Exit
Choose: 0
Bye!
student@ubuntu:~/caesar-lab1$ █

```

Figure 2.6 – Invalid keys and text in Task 1.2

CONCLUSION

In this laboratory work the Caesar cipher and its version with a keyword-permuted alphabet were implemented in Python for the Romanian alphabet of 31 letters. The letters are encoded only by their position in the alphabet from Table 1.1, and the same shift function, $(x + k) \bmod n$ for encryption and $(y - k) \bmod n$ for decryption, serves both tasks.

The classic Caesar cipher has only 30 keys for the Romanian alphabet, so it can be broken in a moment by trying every key and looking for readable text. It gives no real security.

The keyword makes the key space huge, $31! \times 30 \approx 2.47 \times 10^{35}$, so a brute-force attack is no longer possible. However, the cipher remains a simple substitution: every letter is always replaced by the same letter, so the letter frequencies of Romanian are kept in the ciphertext and frequency analysis can still break it. A large key space alone does not make a cipher secure.

The work also showed how important input validation is: the program rejects wrong keys, short keywords and foreign characters with clear messages and asks again, and it accepts both the comma-below and the cedilla forms of Ş and ı, so text typed on different keyboards works.

REFERENCES

1. Laboratory Work No. 1. The Caesar Cipher. Cryptography and Security course materials, UTM, 2026.
2. Stallings W. Cryptography and Network Security: Principles and Practice. 8th ed. Pearson, 2020.
3. Singh S. The Code Book. Anchor Books, 2000.
4. Python 3 documentation. [online]. Available: <https://docs.python.org/3/>

The full source code is available in the GitHub repository:

<https://github.com/Adelina1905/cs-2026>